

Laboratorio 2 – Esquemas de detección y corrección de errores

Descripción

La práctica implementa una aplicación de comunicación entre un cajero automático y un servidor bancario. El sistema sigue una arquitectura de capas con servicios de aplicación, presentación, enlace, simulación de ruido y transmisión. El emisor y el receptor fueron desarrollados en lenguajes distintos: Go para el cajero automático y Python para el servidor bancario.

La capa de aplicación solicita y muestra mensajes, la capa de presentación transforma el texto ASCII en una representación binaria y realiza el proceso inverso en el receptor, la capa de enlace agrega o verifica la información de integridad, la capa de ruido aplica flips probabilísticos sobre cualquier bit de la trama, incluyendo la redundancia, y la capa de transmisión envía y recibe las tramas mediante sockets TCP.

El código de Hamming permite localizar y corregir errores simples mediante bits de paridad ubicados en posiciones específicas de la palabra código (Hamming, 1950). Para la detección de errores se utilizó CRC-32 con el polinomio generador estándar 0x04C11DB7 (Pfeiffer, 2003).

Resultados

Para evaluar el funcionamiento del sistema, se realizaron **800 transmisiones automáticas** entre el cliente desarrollado en Go y el servidor implementado en Python. La matriz experimental estuvo compuesta por 400 transmisiones con Hamming y 400 con CRC-32. Se utilizaron mensajes de 8, 32, 128 y 512 caracteres, probabilidades de error de 0, 0.0001, 0.001, 0.005 y 0.01, y 20 repeticiones por cada combinación.

La implementación final protegió la comunicación en ambas direcciones. Las solicitudes fueron codificadas por el cliente Go y verificadas o corregidas por el servidor Python. Posteriormente, el servidor codificó la respuesta con el mismo algoritmo y el cliente Go realizó su decodificación. En la matriz automática no se aplicó ruido a las respuestas, por lo que las 800 respuestas fueron

recuperadas correctamente. Este resultado permite comprobar la interoperabilidad entre ambos lenguajes, aunque no evalúa el comportamiento del canal de retorno frente al ruido.

1. Tasa de recuperación

En ausencia de ruido, ambos algoritmos recuperaron correctamente el 100 % de los mensajes. Al aumentar la probabilidad de error, se observó una reducción progresiva en la recuperación, aunque Hamming mantuvo porcentajes superiores a CRC-32 en todos los escenarios con ruido.

Probabilidad	CRC-32	Hamming
0	100.00 %	100.00 %
0.0001	87.50 %	100.00 %
0.001	48.75 %	98.75 %
0.005	16.25 %	75.00 %
0.01	7.50 %	57.50 %

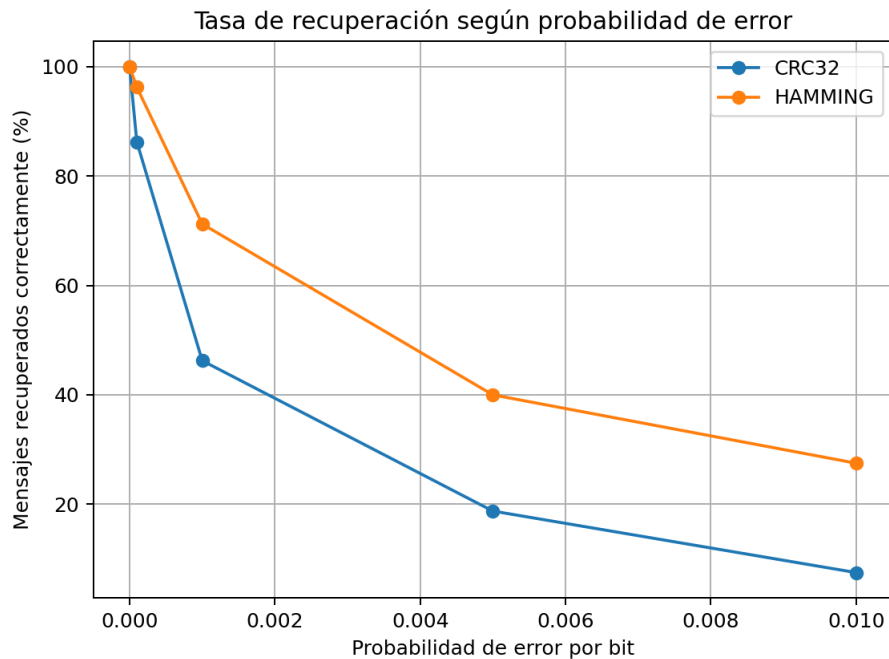


Figura 1. Tasa de recuperación según probabilidad de error

Con una probabilidad de error de 0.0001, Hamming recuperó correctamente el 100 % de los mensajes, mientras que CRC-32 recuperó el 87.50 %. Con una probabilidad de 0.001, Hamming alcanzó una tasa de recuperación de 98.75 %, en comparación con 48.75 % para CRC-32. Para una probabilidad de 0.005, los porcentajes disminuyeron a 75.00 % y 16.25 %, respectivamente. En la probabilidad más alta evaluada, correspondiente a 0.01, Hamming recuperó correctamente el 57.50 % de los mensajes y CRC-32 el 7.50 %.

La diferencia se relaciona con la función de cada algoritmo. Hamming puede corregir errores dentro de cada bloque codificado, mientras que CRC-32 únicamente detecta la corrupción y descarta la trama. Por esta razón, una menor tasa de recuperación para CRC-32 no representa necesariamente un mal funcionamiento. En las pruebas realizadas, todas las tramas aceptadas por CRC-32 contenían el mensaje correcto, mientras que las tramas corruptas fueron rechazadas.

2. Estados reportados por la capa de enlace del receptor

De las 400 transmisiones realizadas con CRC-32, se obtuvieron los siguientes estados:

Estado	Cantidad
ok	208
error	192

Las 208 tramas reportadas como válidas recuperaron correctamente el mensaje original. Las 192 tramas reportadas como error fueron descartadas por el receptor debido a que el CRC calculado no coincidió con el valor recibido. No se observó ninguna trama corrupta aceptada como válida.

Para Hamming se obtuvieron los siguientes resultados:

Estado	Cantidad
ok	200
corrected	167
error	33

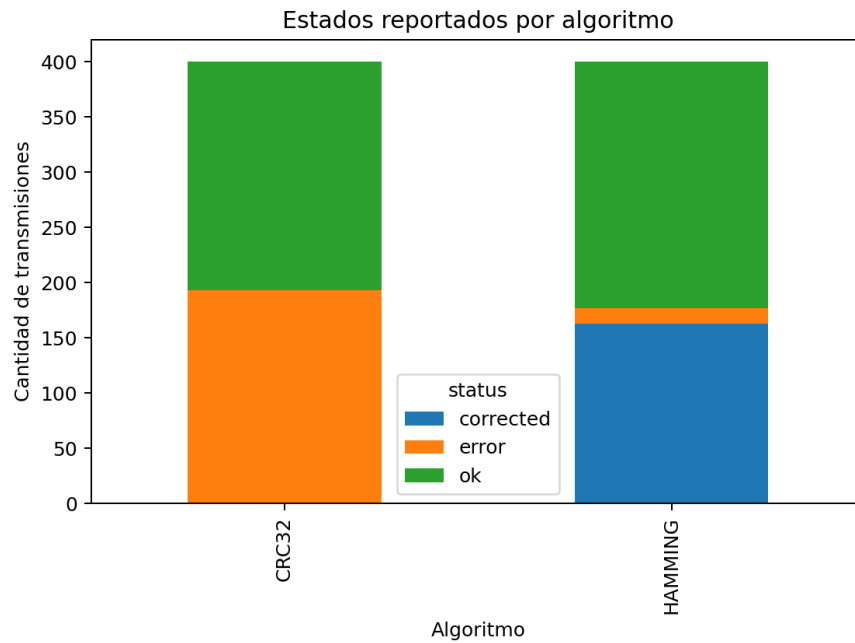


Figura 2. Estados reportados por algoritmo

Las 200 tramas con estado ok no requirieron corrección, mientras que en 167 transmisiones se detectó y corrigió al menos un error dentro de los bloques Hamming. En 33 casos la trama fue considerada no recuperable. En total, Hamming reportó 367 solicitudes procesadas correctamente por la capa de enlace. Sin embargo, en 22 de esas transmisiones el mensaje final no coincidió exactamente con el original, lo que indica que algunos bloques acumularon una cantidad de alteraciones superior a la capacidad de corrección del código.

Por lo tanto, el estado corrected indica que Hamming corrigió errores detectados en uno o más bloques, pero la comparación con el mensaje original siguió siendo necesaria para determinar si la recuperación completa fue correcta.

3. Relación entre ruido y tamaño del mensaje

La cantidad promedio de bits alterados aumentó al incrementar tanto la probabilidad de error como el tamaño del mensaje. Este comportamiento fue esperado, debido a que el ruido se aplicó independientemente sobre cada bit de la trama.

Con una probabilidad de error de 0.01 se observaron aproximadamente 1.18 bits alterados para mensajes de 8 caracteres, 2.93 bits para mensajes de 32 caracteres, 12.90 bits para mensajes de 128 caracteres y 51.83 bits para mensajes de 512 caracteres.

Los resultados evidencian que los mensajes de mayor longitud tienen una mayor probabilidad de acumular múltiples alteraciones. En Hamming, esto incrementa la posibilidad de que dos o más errores ocurran dentro del mismo bloque de doce bits, superando su capacidad de corrección. En CRC-32, cualquier alteración detectada provoca que la trama sea descartada.

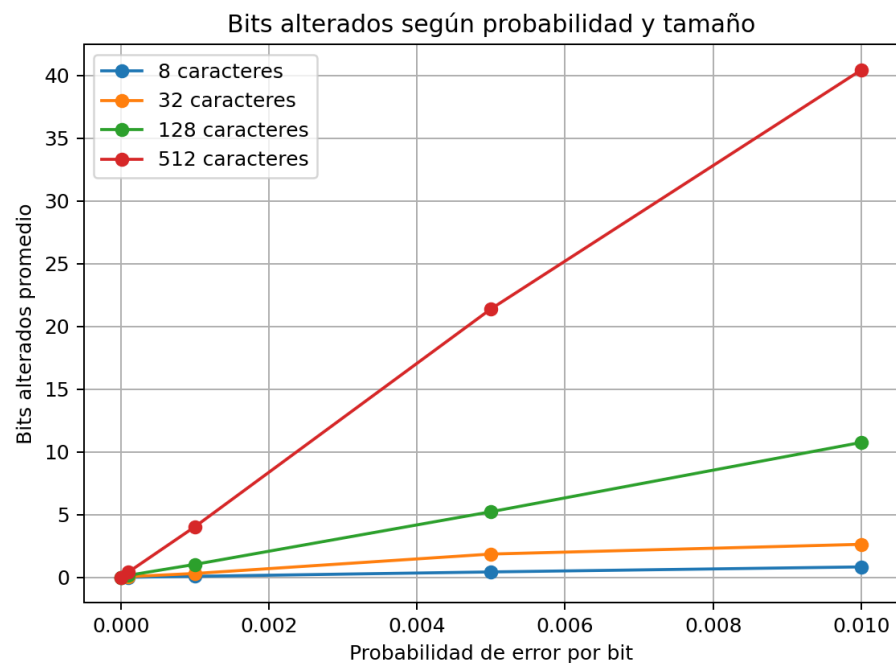


Figura 3. Bits alterados según probabilidad y tamaño

4. Overhead de los algoritmos

El overhead se calculó como la proporción entre la cantidad de bits adicionales y la cantidad de bits originales:

$$\text{Overhead}(\%) = \frac{\text{bits codificados} - \text{bits originales}}{\text{bits originales}} \times 100$$

Los resultados promedio fueron los siguientes:

Tamaño	CRC-32	Hamming
8 caracteres	50.00 %	50.00 %
32 caracteres	12.50 %	50.00 %
128 caracteres	3.13 %	50.00 %
512 caracteres	0.78 %	50.00 %

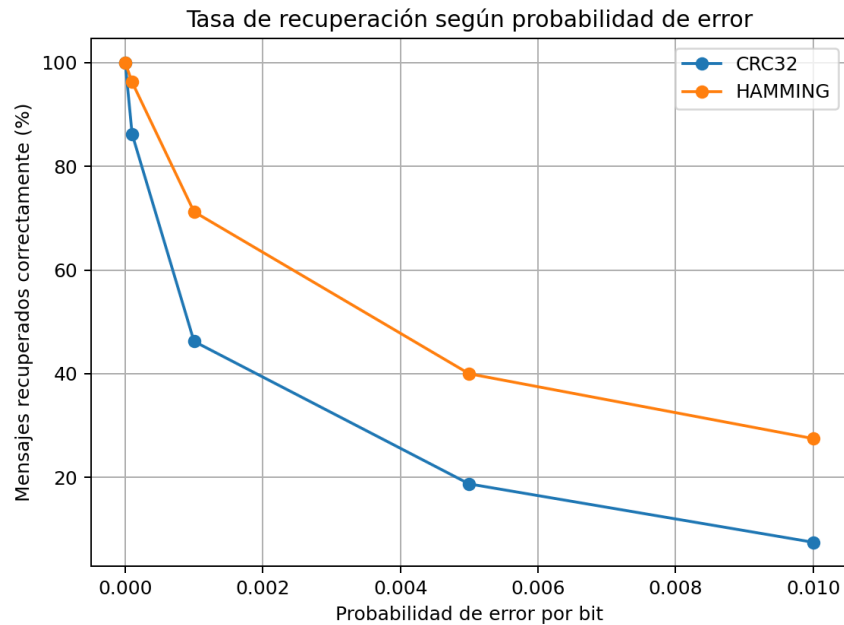


Figura 4. Overhead por tamaño del mensaje

Hamming presentó un overhead constante de 50 %, debido a que cada byte de ocho bits fue codificado como un bloque Hamming de doce bits. Por cada ocho bits originales se agregaron cuatro bits de redundancia.

CRC-32 agregó 32 bits de redundancia por trama, independientemente del tamaño del mensaje. En consecuencia, su overhead porcentual fue elevado para mensajes pequeños, pero disminuyó al aumentar la longitud. En un mensaje de 8 caracteres, los 32 bits adicionales representaron un

overhead de 50 %, mientras que para un mensaje de 512 caracteres representaron únicamente 0.78 %.

Estos resultados muestran un compromiso entre redundancia y capacidad de recuperación. Hamming requiere mayor overhead en mensajes medianos y largos, pero ofrece corrección de errores. CRC-32 utiliza una cantidad fija de redundancia y resulta más eficiente en tramas extensas, aunque necesita retransmisión cuando detecta corrupción.

5. Tiempo de respuesta

El tiempo promedio de recorrido completo fue de aproximadamente 6.74 ms para Hamming y 32.28 ms para CRC-32. Las medianas fueron de 3.93 ms y 25.26 ms, respectivamente.

Estos tiempos incluyen la codificación de la solicitud, el envío por socket, la verificación o corrección en el servidor, la codificación de la respuesta y su decodificación en el cliente. Aunque Hamming presentó menores tiempos en esta ejecución, estos resultados dependen de la implementación, el equipo, el sistema operativo y las características de los mensajes. Por ello, no deben interpretarse como una comparación universal entre ambos algoritmos.

Discusión

1 ¿Qué algoritmo tuvo un mejor funcionamiento?

No se identificó un algoritmo superior en todos los aspectos, debido a que Hamming y CRC-32 persiguen objetivos diferentes.

Hamming presentó una mayor tasa de recuperación de mensajes bajo condiciones de ruido. Con una probabilidad de error de 0.001 recuperó correctamente el 98.75 % de los mensajes, mientras que CRC-32 recuperó el 48.75 %. Incluso con una probabilidad de 0.01, Hamming conservó una tasa de recuperación de 57.50 %, en comparación con 7.50 % para CRC-32.

Este comportamiento se explica por la implementación de Hamming por bloques de ocho bits de datos y doce bits codificados. Cada bloque puede corregir un error, por lo que una misma trama

puede recuperarse aunque existan varios errores, siempre que no se concentren más de uno dentro del mismo bloque.

CRC-32 mostró un mejor comportamiento cuando el criterio fue evitar que información corrupta llegara a la capa de aplicación. Todas las tramas aceptadas recuperaron correctamente el mensaje original y las 192 tramas alteradas detectadas fueron descartadas. No se observaron falsos positivos en los que una trama corrupta fuera aceptada como válida.

Por lo tanto, Hamming tuvo un mejor desempeño en términos de disponibilidad y recuperación inmediata. CRC-32 tuvo un mejor desempeño en términos de detección segura y prevención del procesamiento de datos alterados.

2 ¿Qué algoritmo fue más flexible ante mayores tasas de error?

Hamming fue más flexible para mantener la disponibilidad del mensaje a medida que aumentó la tasa de error. Su capacidad de corregir un error por bloque permitió recuperar una proporción considerable de las tramas incluso con probabilidades relativamente altas.

Sin embargo, esta flexibilidad tuvo un costo de redundancia constante del 50 %. Además, cuando dos o más errores ocurrieron dentro del mismo bloque Hamming, la corrección dejó de estar garantizada. Esto explica que 22 solicitudes fueran procesadas como recuperadas por la capa de enlace, pero no coincidieran finalmente con el mensaje original.

CRC-32 fue más flexible para detectar corrupción, pero no para recuperar información. Su comportamiento fue conservador: ante una diferencia en el residuo, la trama fue descartada. A medida que aumentó la probabilidad de error y el tamaño de la trama, también aumentó la frecuencia de rechazo.

En consecuencia, la flexibilidad depende del criterio utilizado. Hamming resulta más flexible cuando se necesita continuar la comunicación sin retransmitir ante errores aislados. CRC-32 resulta más flexible cuando la prioridad es detectar cualquier alteración y existe un mecanismo para solicitar nuevamente la trama.

3 ¿Cuándo es mejor utilizar detección en lugar de corrección?

La detección resulta preferible cuando la integridad de la información es más importante que su disponibilidad inmediata y existe la posibilidad de retransmisión.

En una aplicación bancaria, una solicitud alterada no debería procesarse si existe incertidumbre sobre su contenido. Por ejemplo, un cambio en los bits correspondientes al monto de un retiro podría modificar el valor solicitado. En ese escenario sería más seguro descartar la trama mediante CRC-32 y solicitar una retransmisión que intentar recuperar información cuya corrección no pueda garantizarse.

La corrección resulta conveniente cuando:

- la retransmisión es costosa o imposible;
- la latencia de repetir el envío es elevada;
- se espera una cantidad baja de errores aislados;
- el sistema puede aceptar el overhead adicional;
- se desea mantener la disponibilidad del mensaje.

La detección resulta conveniente cuando:

- la integridad es prioritaria;
- pueden ocurrir errores múltiples;
- existe un mecanismo de retransmisión;
- una corrección incorrecta tendría consecuencias importantes;
- se necesita menor overhead relativo en mensajes extensos.

4. Efecto del overhead

La implementación de Hamming presentó un overhead constante de 50 %. Esta redundancia permitió corregir un error por bloque de doce bits y fue una de las razones de su mayor tasa de recuperación.

CRC-32 agregó únicamente 32 bits por trama. Su overhead fue igual al de Hamming para mensajes de ocho caracteres, pero disminuyó considerablemente para mensajes más largos. En mensajes de 512 caracteres, CRC-32 utilizó un overhead de 0.78 %, mientras que Hamming mantuvo 50 %.

Esto evidencia un compromiso claro. Hamming ofrece mayor capacidad de recuperación a cambio de transmitir más información. CRC-32 utiliza menos redundancia en tramas largas, pero requiere retransmitir cuando detecta corrupción.

5. Comunicación protegida bidireccional

La implementación final permitió que ambos programas actuaran como emisores y receptores. Go codificó las solicitudes y Python las verificó o corrigió. Después, Python codificó las respuestas y Go realizó su verificación y decodificación.

Las 800 respuestas fueron recuperadas correctamente, lo que confirmó la compatibilidad de Hamming y CRC-32 entre ambos lenguajes. No obstante, la matriz automática aplicó ruido únicamente a las solicitudes. Por ello, los resultados del canal de retorno demuestran interoperabilidad, pero no permiten evaluar el desempeño de los algoritmos ante errores en las respuestas.

Una evaluación futura podría aplicar probabilidades de ruido independientes en ambos sentidos y comparar la recuperación de solicitudes **y respuestas**.

6. Limitaciones de las pruebas

Los resultados dependen de las configuraciones utilizadas y de la naturaleza aleatoria del ruido. Aunque cada combinación se repitió 20 veces, un número mayor de repeticiones permitiría obtener porcentajes más estables.

También deben considerarse las siguientes limitaciones:

- se evaluaron únicamente cuatro tamaños de mensaje;
- se utilizaron cinco probabilidades de error;
- los errores se generaron de forma independiente;
- no se simulaban errores en ráfaga;
- no se aplicó ruido a las respuestas durante la matriz automática;
- CRC-32 no implementó retransmisión automática;

- Hamming puede fallar cuando un bloque contiene varios errores;
- los tiempos obtenidos dependen del entorno local de ejecución.

A pesar de estas limitaciones, las pruebas permitieron comparar de manera clara el comportamiento de un mecanismo de detección y uno de corrección.

Comentario grupal

El desarrollo del laboratorio permitió comprender que la comunicación entre dos aplicaciones requiere algo más que el envío de bytes mediante sockets. Fue necesario definir una arquitectura por capas que separara la construcción del mensaje, su representación binaria, la adición de redundancia, la simulación del ruido y la transmisión.

La implementación en Go y Python también evidenció la importancia de definir un protocolo común. Ambos programas debieron utilizar la misma estructura JSON, el mismo orden de bits, las mismas posiciones de paridad y el mismo polinomio CRC-32. Una diferencia entre las convenciones de ambos lenguajes habría impedido la correcta interpretación de las tramas.

La implementación bidireccional permitió comprobar que cada programa podía desempeñar funciones de emisor y receptor. El cliente Go no solo codificó solicitudes, sino que también verificó y decodificó respuestas. De forma equivalente, el servidor Python recibió y procesó solicitudes protegidas y posteriormente codificó sus propias respuestas.

Las pruebas masivas permitieron observar comportamientos que no eran evidentes en una ejecución manual. Hamming mostró una alta capacidad de recuperación gracias a la codificación por bloques, pero también presentó casos en los que varios errores dentro de un mismo bloque impidieron recuperar exactamente el mensaje. CRC-32, por otra parte, no recuperó las tramas alteradas, pero evitó que mensajes corruptos fueran aceptados como válidos.

Conclusiones

- La arquitectura implementada permitió comunicar un cliente desarrollado en Go con un servidor desarrollado en Python mediante sockets TCP, separando las responsabilidades de aplicación, presentación, enlace, ruido y transmisión.
- La protección de integridad se implementó en ambas direcciones. Go codificó las solicitudes y decodificó las respuestas, mientras que Python verificó las solicitudes y codificó las respuestas con el mismo algoritmo.
- En ausencia de ruido, Hamming y CRC-32 recuperaron correctamente el 100 % de los mensajes, lo que confirmó la compatibilidad de las implementaciones entre ambos lenguajes.
- Hamming presentó una mayor tasa de recuperación bajo condiciones de ruido. Con una probabilidad de error de 0.001 recuperó correctamente el 98.75 % de los mensajes, mientras que CRC-32 recuperó el 48.75 %.
- Con una probabilidad de error de 0.01, Hamming recuperó correctamente el 57.50 % de los mensajes y CRC-32 el 7.50 %. Esto demostró la ventaja de la corrección por bloques cuando los errores se distribuyen entre diferentes palabras código.
- CRC-32 presentó un comportamiento confiable como algoritmo de detección. Las 208 tramas reportadas como válidas recuperaron correctamente el mensaje y las 192 tramas detectadas como corruptas fueron descartadas.
- Hamming procesó correctamente 367 solicitudes desde la perspectiva de la capa de enlace, pero 22 mensajes no coincidieron finalmente con el original. Esto confirmó que varios errores dentro de un mismo bloque pueden exceder la capacidad de corrección del código.
- Hamming presentó un overhead constante de 50 %, debido a la codificación de cada byte de ocho bits como un bloque de doce bits. Esta redundancia permitió corregir un error por bloque.
- CRC-32 agregó 32 bits fijos por trama. Su overhead disminuyó de 50 % para mensajes de ocho caracteres a 0.78 % para mensajes de 512 caracteres, por lo que resultó más eficiente en mensajes largos.

- La cantidad promedio de bits alterados aumentó con el tamaño del mensaje y la probabilidad de error. Con una probabilidad de 0.01, los mensajes de 512 caracteres presentaron aproximadamente 51.83 bits alterados en promedio.
- Hamming resultó más apropiado cuando se requería recuperar mensajes sin retransmisión y se esperaban errores aislados por bloque. CRC-32 resultó más apropiado cuando la integridad era prioritaria y existía la posibilidad de retransmitir.
- Las 800 respuestas fueron recuperadas correctamente, lo que validó la interoperabilidad bidireccional. Sin embargo, debido a que no se aplicó ruido a las respuestas, se recomienda evaluar en el futuro el canal de retorno bajo las mismas probabilidades de error.
- Como mejoras futuras se recomienda implementar retransmisión automática para CRC-32, evaluar errores en ráfaga y aplicar ruido configurable tanto a las solicitudes como a las respuestas.

Referencias

Eddy, W. (2022). Transmission Control Protocol (TCP) (RFC 9293). RFC Editor.
<https://doi.org/10.17487/RFC9293>

Hamming, R. W. (1950). Error detecting and error correcting codes. Bell System Technical Journal, 29(2), 147-160. <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>

Pfeiffer, S. (2003). The Ogg encapsulation format version 0 (RFC 3533). RFC Editor.
<https://www.rfc-editor.org/rfc/rfc3533.html>

Python Software Foundation. (n.d.). socket — Low-level networking interface. Python documentation. Recuperado el 4 de agosto de 2026, de <https://docs.python.org/3/library/socket.html>

Ross, S. M. (2014). A first course in probability (9th ed.). Pearson.

Link al repositorio de Github: https://github.com/tsc221645/redes_lab_2.git